

TP : Prise en main de Matlab/Simulink

Projet 1

Analyse temporelle d'un signal carré

Dans ce projet, vous placez deux boîtes de la librairie Simulink et observez l'évolution temporelle d'un signal carré.

 Réglages

- Ouvrez la librairie `Simulink Browser` puis placez les boîtes sur l'interface :
 - `Simulink > Sources > Pulse generator`
 - `Simulink > Sinks > Scope`
- Reliez les boîtes
- Double cliquez sur la boîte `Pulse generator` pour mettre le paramètre `Pulse type` sur la valeur `Sample based` et le paramètre `Sample Time` à 0.1 seconde.

 Information sur `Sample Time` 

- Vous réglez le paramètre `Stop time` sur la valeur 10 (bandeau haut)
- Lancez la simulation

config : pulse generator:

Block Parameters: Pulse Generator

×

else
Y(t) = 0
end

Pulse type determines the computational technique used.

Time-based is recommended for use with a variable step solver, while Sample-based is recommended for use with a fixed step solver or within a discrete portion of a model using a variable step solver.

Parameters

Pulse type: Sample based

Time (t): Use simulation time

Amplitude:
1

Period (number of samples):
20

Pulse width (number of samples):
10

Phase delay (number of samples):
0

Sample time:
0.1

☒ Interpret vector parameters as 1-D

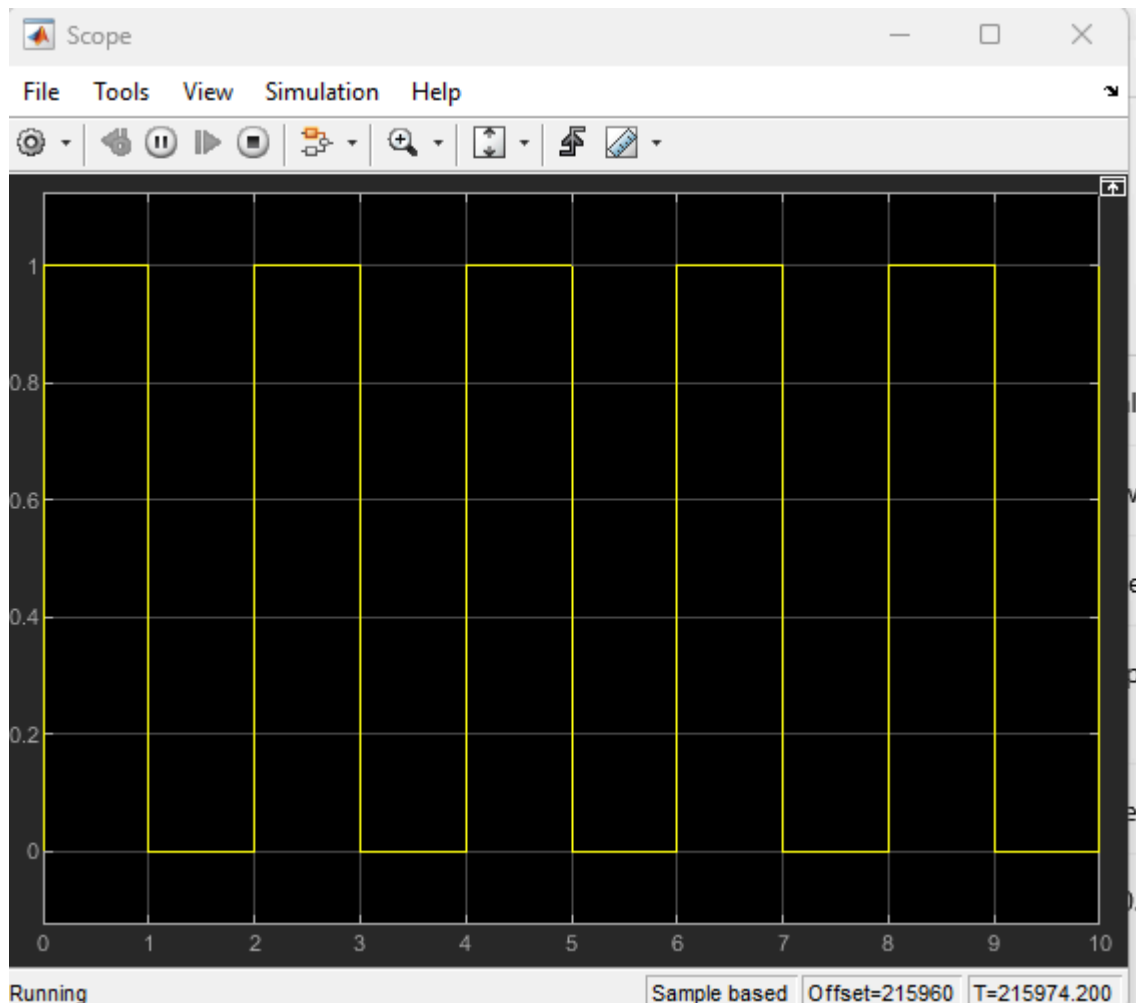
?

OK

Cancel

Help

Apply



1. Quelle est la durée de la simulation ?

Durée de la simulation = Stop time = 10 secondes

C'est la durée totale pendant laquelle Simulink fait évoluer le signal.

2. Quelle est la durée du signal carré à l'état haut ?

Le paramètre **Pulse width (%) = 50** signifie que le signal est à 1 pendant **50 % de la période**.

$$\text{Durée à l'état bas} = 50\% \times 2 \text{ s} = 1 \text{ s}$$

Durée à l'état haut = 1 seconde

3. Quelle est la durée du signal carré à l'état bas ?

L'autre moitié de la période correspond à l'état bas (0).

$$\text{Durée à l'état bas} = 50\% \times 2 \text{ s} = 1 \text{ s}$$

Durée à l'état bas = 1 seconde

4. Quelle est la période du signal carré ?

C'est le temps total d'un cycle complet (haut + bas) :

un cycle complet (haut + bas)

$$\text{Période} = 2 \text{ s}$$

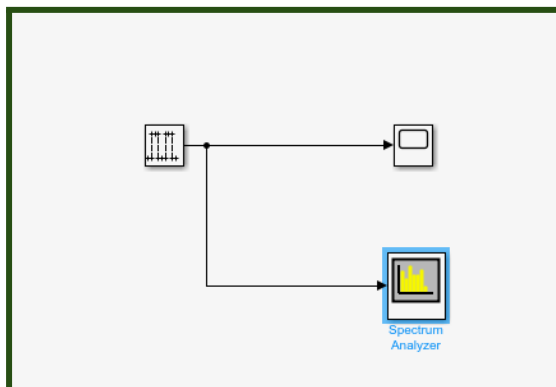
Période du signal carré = 2 secondes

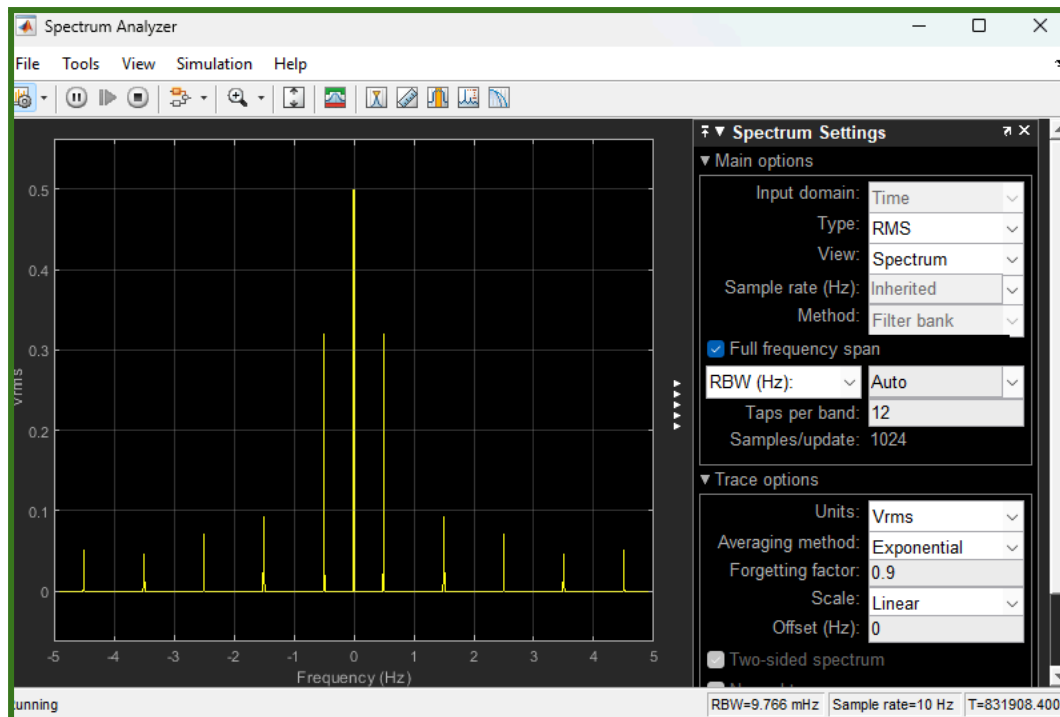
Analyse fréquentielle du signal carré

Dans cette partie, vous observez l'évolution fréquentielle du signal carré.

Réglages

- Ajouter une boîte `DSP System Toolbox > Sinks > Spectrum analyser`
- Reliez les boîtes `Pulse generator` et `Spectrum analyser`
- Double cliquez sur la boîte `Spectrum analyser` et dans le menu `View > Spectrum Settings` mettre le paramètre `Type` sur `RMS`.
- Vous réglez le paramètre `Stop time` sur la valeur `inf` (infinie)
- Lancer la simulation





1. Relevez la fréquence maximale de l'axe des fréquences. Que représente cette valeur ?

Dans le **Spectrum Analyzer**, tu vois que l'axe des fréquences s'arrête à une certaine valeur — par exemple **5 Hz**.

Cette valeur correspond à la **fréquence de Nyquist** :

$F_{max} \Rightarrow f_e / 2 \implies$ avec f_e est la **fréquence d'échantillonnage**.

La fréquence maximale de l'axe est 5 Hz. donc **valeur** correspond à la fréquence de Nyquist, c'est-à-dire la plus haute fréquence que l'on peut représenter sans aliasing quand on échantillonne à 10 Hz.

2. Est-ce en accord avec la période d'échantillonnage du **Pulse generator**?

Oui

c'est en accord avec la période d'échantillonnage :

$$T_e = 0.1 \text{ s} \Rightarrow f_e = 10 \text{ Hz} \Rightarrow f_{max} = 5 \text{ Hz}$$

3. Que représente la raie à la fréquence de 0 Hz ?

La raie à **0 Hz** représente la **valeur moyenne** du signal carré (composante continue, ici 0.5)

Générer une séquence binaire aléatoire

Lors d'une transmission numérique, les données sont aléatoires. Dans cette partie, nous complétons notre projet en ajoutant une boîte générant des valeurs aléatoires (ou presque !).

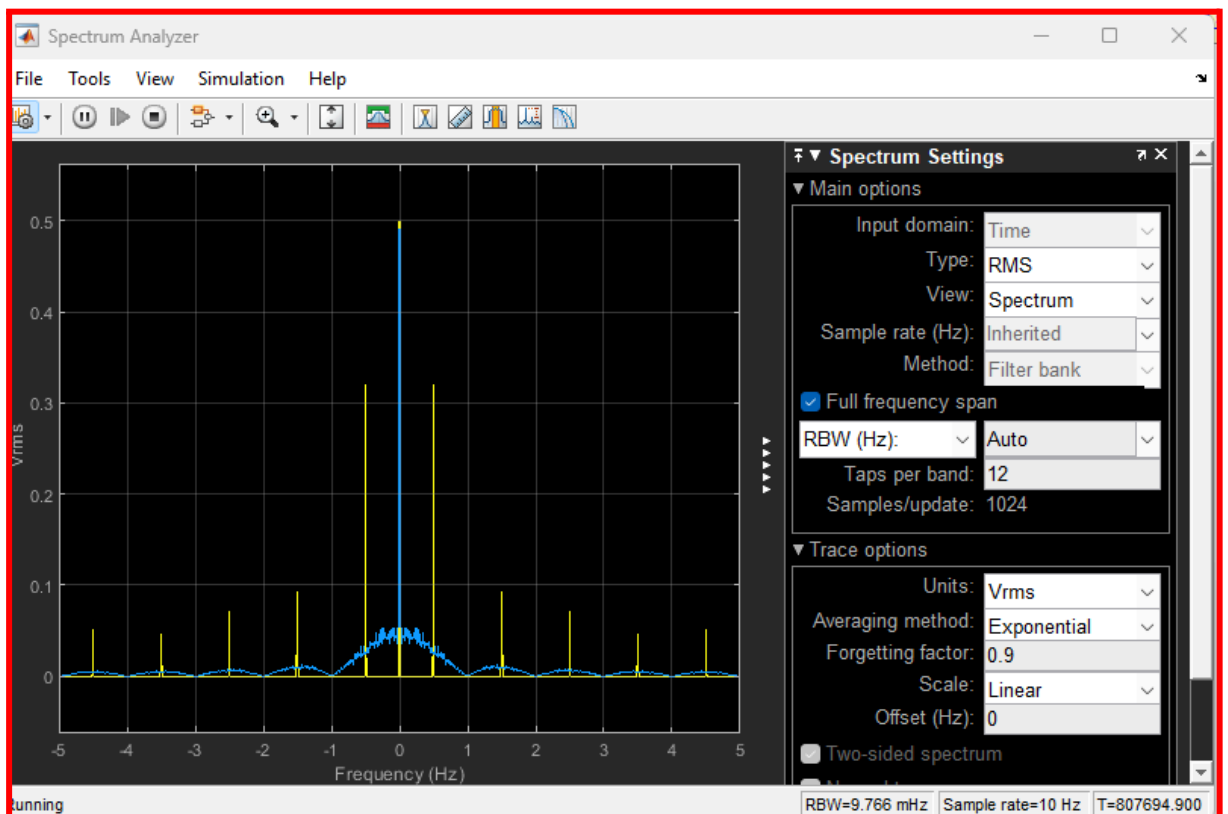
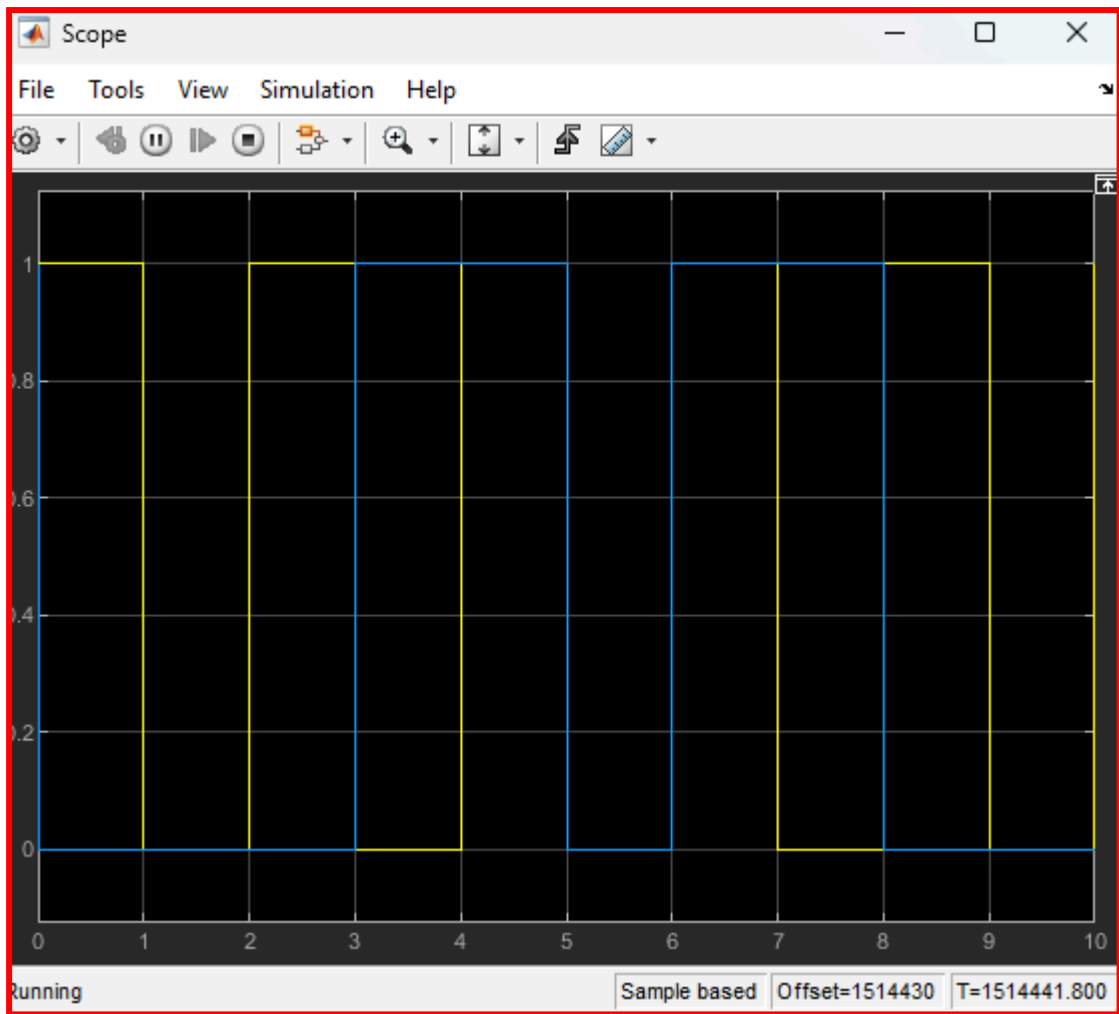
Dans cet exemple, les observations temporelles et fréquentielle sont réalisées.

 **Réglages**

- Ajouter une boîte `Communications Toolbox > Comm Sources > Random Data Sources > Bernoulli Binary`. Ne modifiez pas la valeur des paramètres.
- Ajouter une boîte `Simulink > Discrete > Zero-Order Hold` entre le générateur et la boîte `Spectrum Analyzer`.
- Double cliquez sur la boîte et fixez la valeur du paramètre `Sample Time` à 0,1 seconde.

 **Information sur la boîte `Zero-Order Hold`** 

- Double cliquez sur la boîte `Scope` dans le menu `View > Configuration Properties` dans l'onglet `main` mettre la valeur du paramètre `Number of input ports` à 2. Appuyer sur le bouton `Layout`, choisir une superposition des deux traces.
- Double cliquez sur la boîte `Spectrum analyzer`. Dans le menu `File > Number of Inputs Ports` mettre le paramètre à la valeur 2.
- Connecter les boîtes entre elles.
- Fixer la durée de simulation à 10 secondes et lancer la simulation et comparer les traces temporelles.
- Fixer la durée de simulation à inf et lancer la simulation.



1. Quelle est la différence entre les deux spectres ?

- **Spectre du signal carré** (déterministe)
- **Spectre du signal aléatoire** (aléatoire, bruit)

Spectre du signal carré :

Il est **discret** : on observe des **pics (raies)** à des fréquences bien précises.

Ces pics correspondent aux **harmoniques impaires** du signal carré :
 $f, 3f, 5f, 7f, \dots, f, 3f, 5f, 7f, \dots$

L'amplitude de chaque raie diminue quand la fréquence augmente.

Le spectre est donc **structuré et périodique**.

Spectre du signal aléatoire :

Il est **continu** : il n'y a **pas de raies** mais une **distribution d'énergie sur une large bande**.

Le spectre ressemble à une **zone pleine** (ou plate si c'est du bruit blanc).

Le signal n'a pas de périodicité, donc il contient **toutes les fréquences** dans une certaine bande.

- Le **signal aléatoire** a un spectre continu (énergie répartie sur plusieurs fréquences sans périodicité).
- * Le **signal carré** a un spectre discret (harmoniques impaires régulières).

2. Relever l'amplitude et la fréquence des différentes raies et comparer les valeurs avec la série de Fourier d'un signal carré :

Les fréquences des raies observées correspondent bien à la série de Fourier (harmoniques impaires).

Leurs amplitudes décroissent proportionnellement à $1/(2n+1)$ / $1/(2n+1)$.

3. Relever les fréquences pour lesquelles le spectre du signal aléatoire s'annule.
Donner alors la bande passante du signal.

Le signal aléatoire (souvent un bruit) a un spectre continu.

Il s'annule au-delà d'une certaine fréquence limite, appelée fréquence de coupure f_{cf_cfc} .

La bande passante (BP) correspond à l'intervalle où l'énergie du signal est présente.

Le spectre du signal aléatoire s'annule au-delà de la fréquence de coupure.

La bande passante est donc comprise entre 0 Hz et f_{cf_cfc} (ou jusqu'à $f_e/2$ / $2f_e/2$ pour un bruit blanc échantillonné).

4. Quelle est la particularité du signal généré aléatoirement ?

Un signal aléatoire : n'est pas périodique ses valeurs varient de manière imprévisible il contient de nombreuses fréquences (voire toutes dans une bande donnée) son spectre est continu, souvent plat s'il s'agit d'un bruit blanc

Les codes en ligne

Code NRZ

Mise en oeuvre

C'est le code le plus simple. Pour chaque valeur binaire on associe une valeur de tension :

- "0" logique -> -V
- "1" logique -> +V

Pour visualiser correctement le signal, on fixe la valeur $V=+1$ V.

Préparation du projet 2

Faire une copie du fichier [TP1Projet1](#) vers [TP1Projet2](#).

Dans cette partie, vous devez construire les schémas à partir des images fournies.

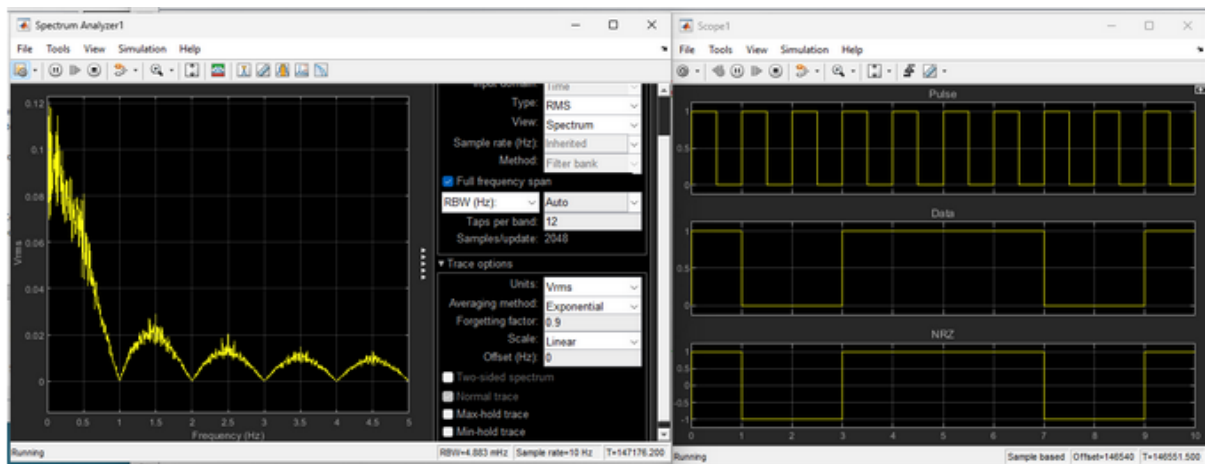
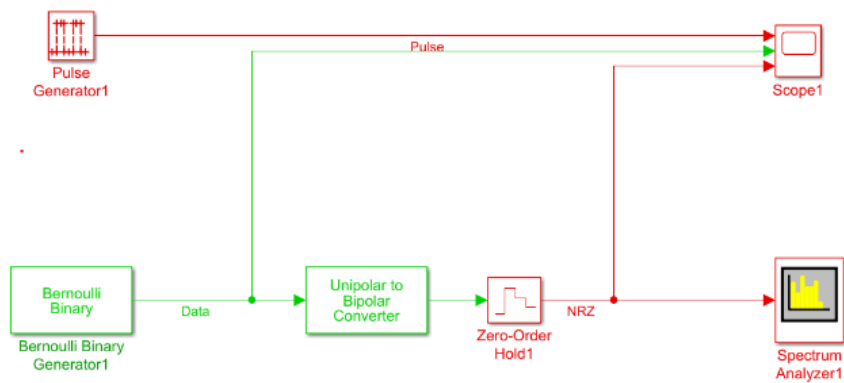
Le schéma de départ est le suivant :

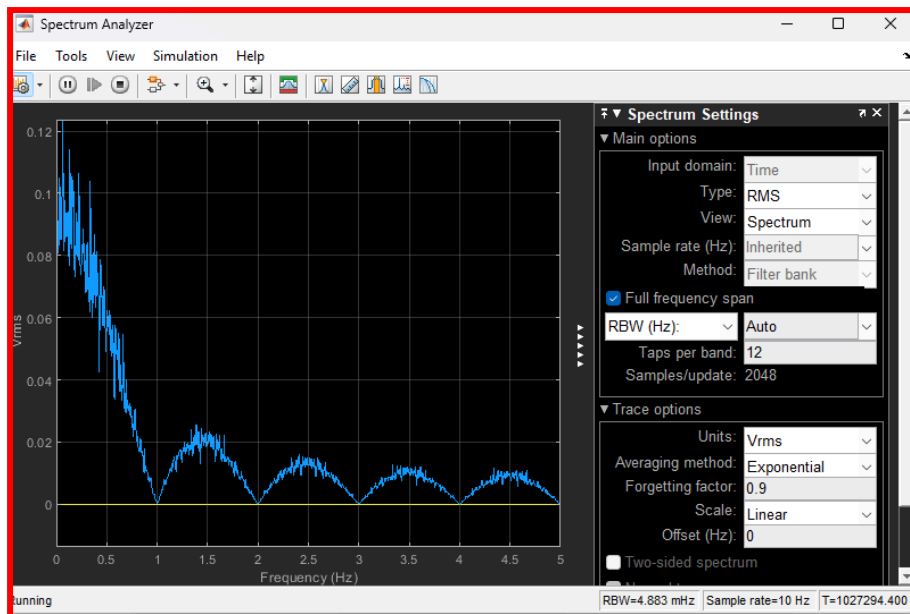
Étude du code

Réglages

- Insérez la boîte Communications Toolbox > Utility Blocs > Unipolar to bipolar Converter
- Double cliquez sur la boîte et mettre la valeur du paramètre M-ary number à 2.
- Double cliquez sur la boîte Spectrum Analyzer dans le menu View > Spectrum sttoings dans l'onglet Trace options décochez la case Two-Sided spectrum
- Fixer la durée de simulation Stop Time à inf
- Lancer la simulation

Schéma Résultats de simulation





1. Mesurer la bande passante du signal NRZ.

Un **signal NRZ** (Non Return to Zero) est un signal binaire qui reste :

- à **+A** pendant toute la durée d'un bit logique "1"
- à **-A** ou 0 pendant toute la durée d'un bit logique "0"

il ne revient pas à zéro entre deux bits, d'où son nom.

La bande passante du signal NRZ est d'environ $R_b/2R_{b_b} / 2R_b/2$, soit la moitié du débit binaire.

C'est donc un **signal en créneaux**, mais dont les transitions dépendent du contenu binaire (aléatoire ou pseudo-aléatoire).

2. Quel est le défaut majeur du signal NRZ ?

Bande passante : environ $R_b/2R_{b_b} / 2R_b/2$.

Défaut majeur : absence de transitions dans le cas de suites longues de bits identiques → perte de synchronisation et présence d'une composante continu

Code RZ unipolaire

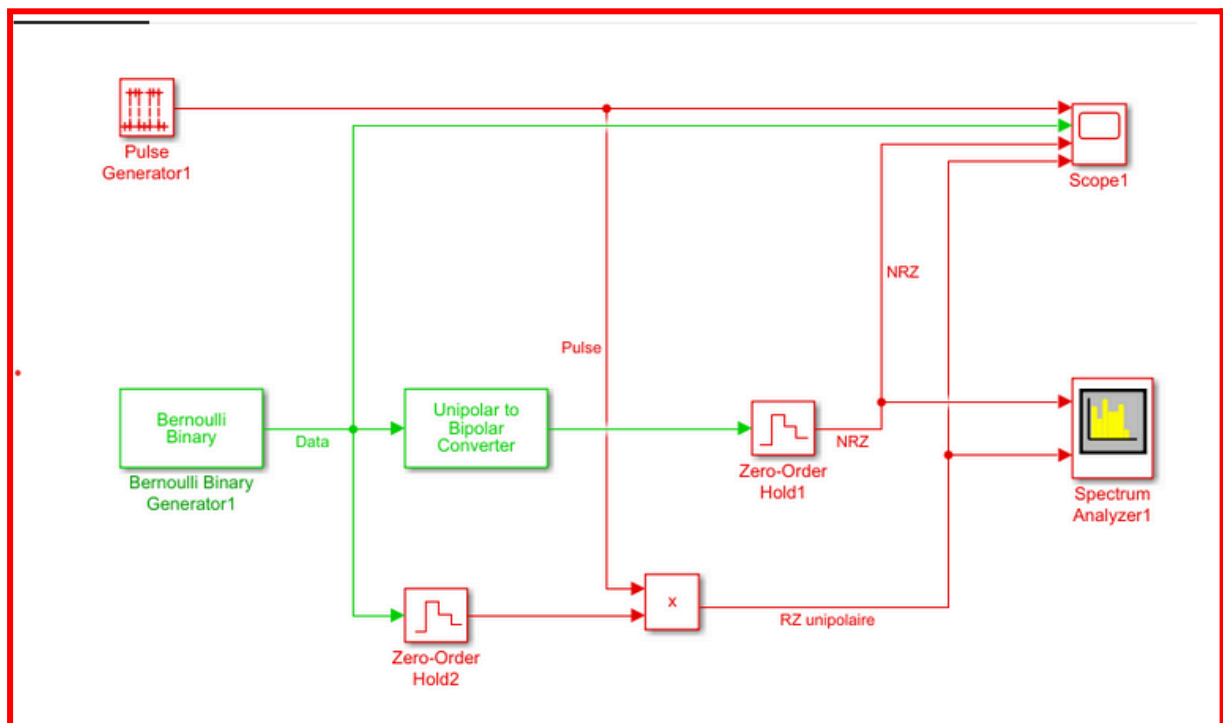
Mise en oeuvre

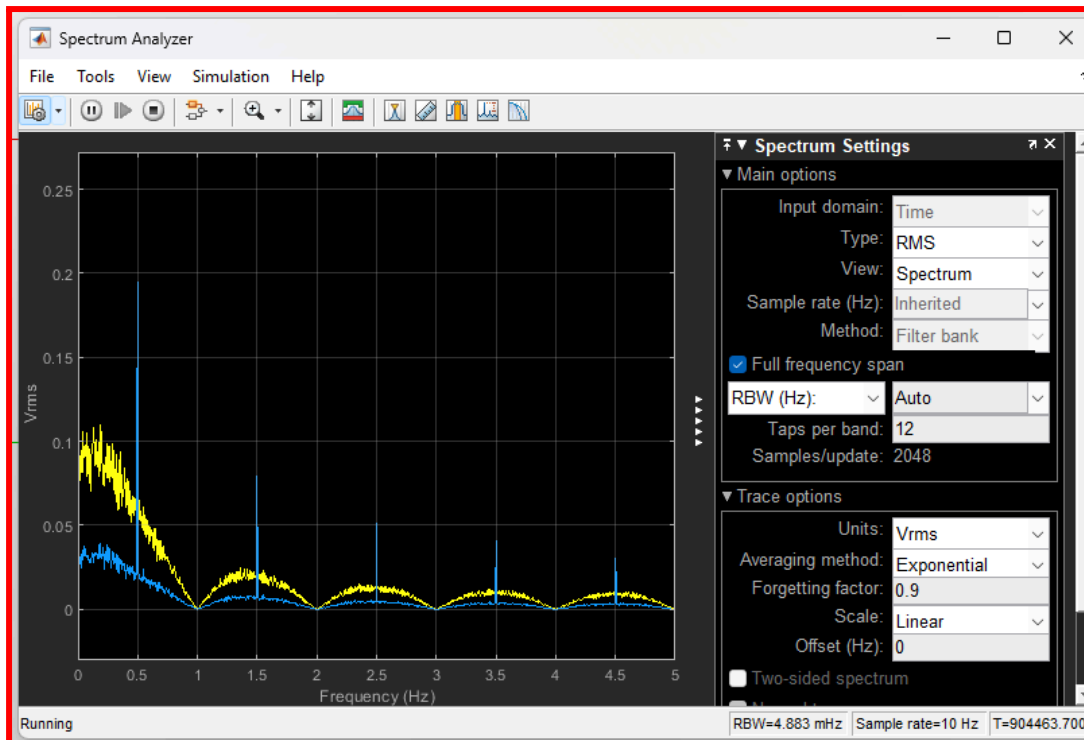
Pour créer ce code il faut ajouter un générateur d'horloge, **Pulse Generator**, de période égale à la durée d'un bit et multiplier le signal d'horloge avec le code binaire généré par la boîte **Bernouilli Binary**.



Réglages

- Insérez la boîte Simulink > Math Operations > Product
- Fixer la durée de simulation Stop Time à la valeur inf





1. Quelle est la particularité du spectre du signal RZ unipolaire ?

Pas de composante continue (pas de raie à 0 Hz) et deux raies principales symétriques.

2. Relevez l'amplitude et la fréquence des deux raies.

$$f = R_b \text{ et } f = 2R_b$$

Deux pics principaux à environ $f = R_b$ et $f = 2R_b$, la première plus forte que la seconde.

3. Quelle est la bande passante du signal RZ unipolaire ?

La bande passante du signal RZ unipolaire est environ de R_b à $2R_b$, soit deux fois plus large que celle du NRZ.

4. A quoi peuvent servir les deux raies du spectre ?

Les deux raies principales du spectre représentent les composantes périodiques dominantes du signal.

Elles peuvent être utilisées pour :

1. Synchroniser le récepteur (horloge) : les raies permettent d'extraire un signal de référence pour caler l'échantillonnage.
2. Identifier la fréquence fondamentale du codage.

3. Filtrage sélectif : elles permettent de concevoir un filtre passe-bande pour isoler la bande utile.

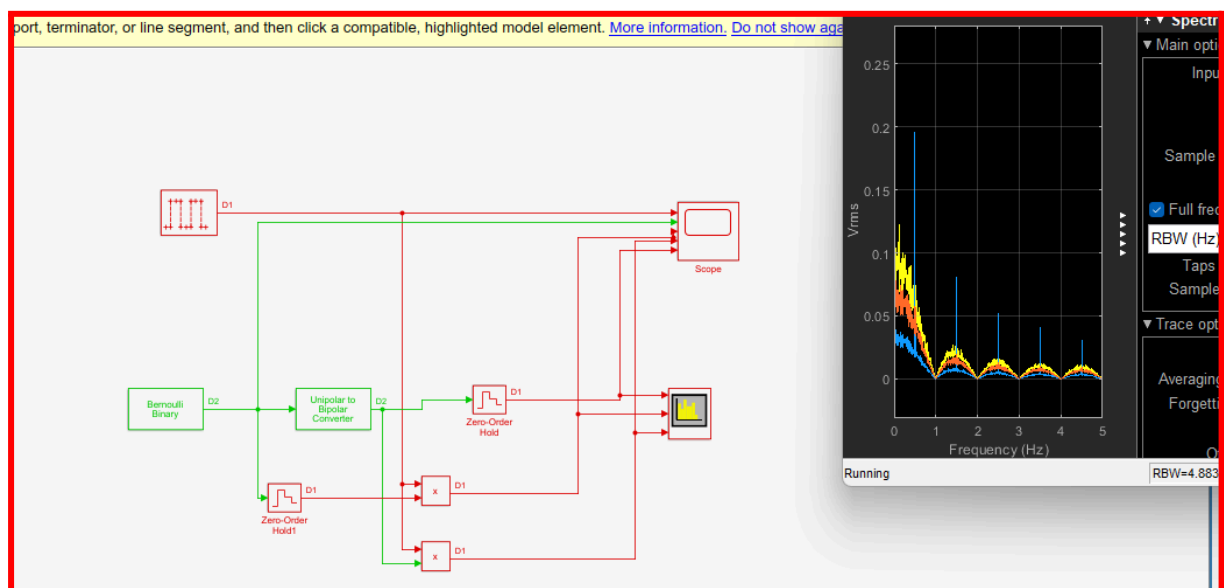
Code RZ bipolaire

Mise en oeuvre

Pour créer ce code, il faut réaliser la multiplication entre le code NRZ et le signal **Pulse**.

 **Réglages**

- Insérez la boîte Simulink > Math Operations > Product
- Relier les boîtes comme sur la figure
- Fixer la durée de simulation Stop Time à la valeur inf



1. Donner la bande passante du code RZ bipolaire.

Environ **Rb**(semblable au RZ unipolaire), donc **deux fois plus large que le NRZ**.

2. Quelle est l'amplitude du spectre à 0 Hz ?

Nulle (0) — le signal RZ bipolaire ne contient **aucune composante continue**, car les impulsions positives et négatives s'annulent en moyenne.

Le code Manchester

Mise en oeuvre

Pour réaliser le code Manchester, il faut réaliser l'opération logique OU exclusif (XOR) entre l'horloge (Pulse) et le message binaire à coder (Bernoulli Binary). La figure montre le codage d'un 1 et 0 logiques.

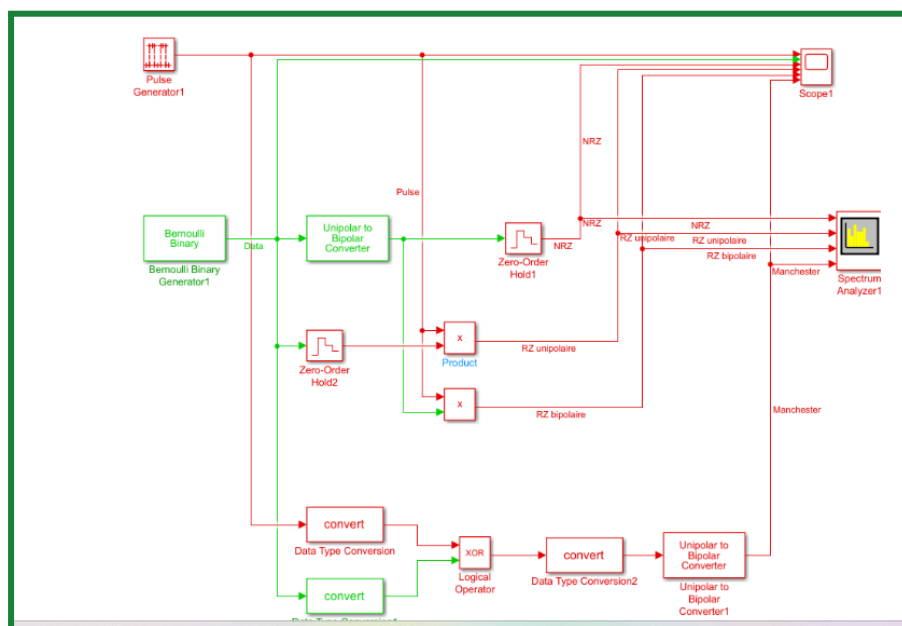
Réglages

- Insérez trois boîtes `Simulink > Commonly Used Blocks > Convert`

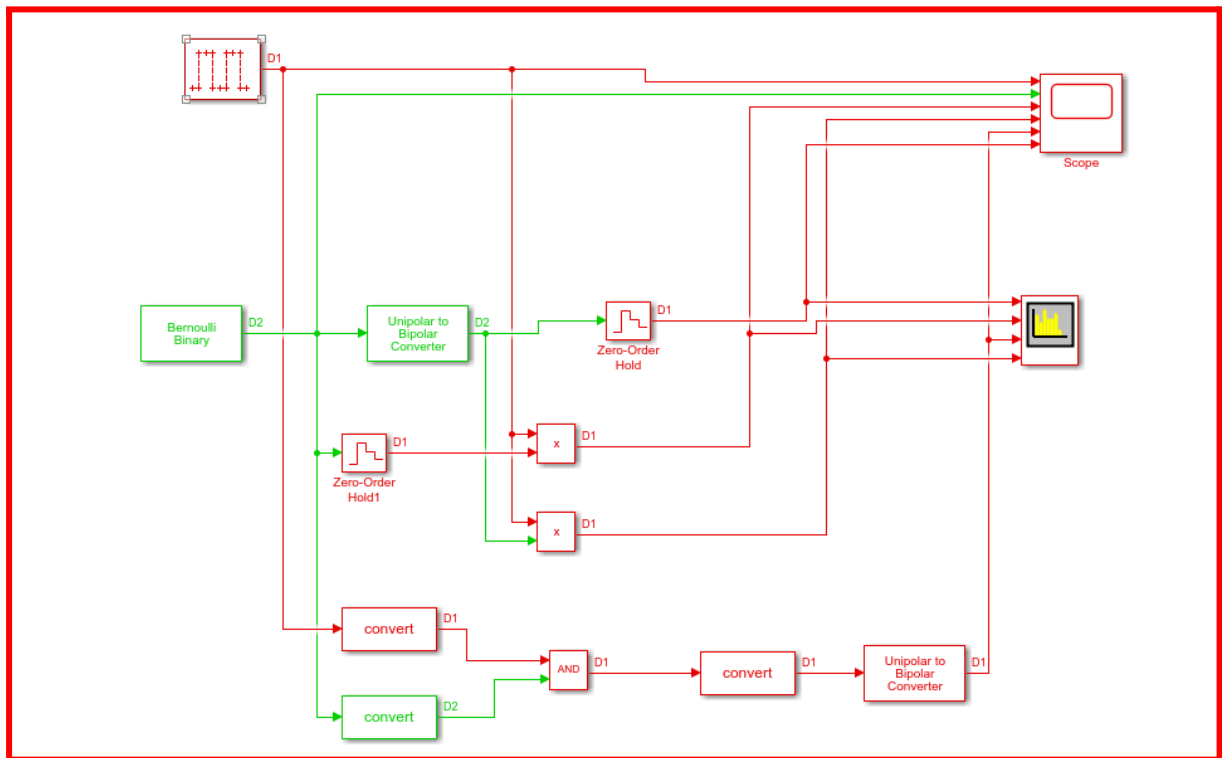
 **Pourquoi utiliser les boîtes `Convert` ?** 

- Insérez une boîte `Simulink > Logic and Bit Operations > Logical Operator`. Double cliquez sur la boîte et choisir l'opérateur booléen `XOR`.
- Insérez la boîte `Communications Toolbox > Utility Blocs > Unipolar to bipolar Converter`
- Fixer la durée de simulation `Stop Time` à la valeur 10 secondes et observer les chronogrammes.

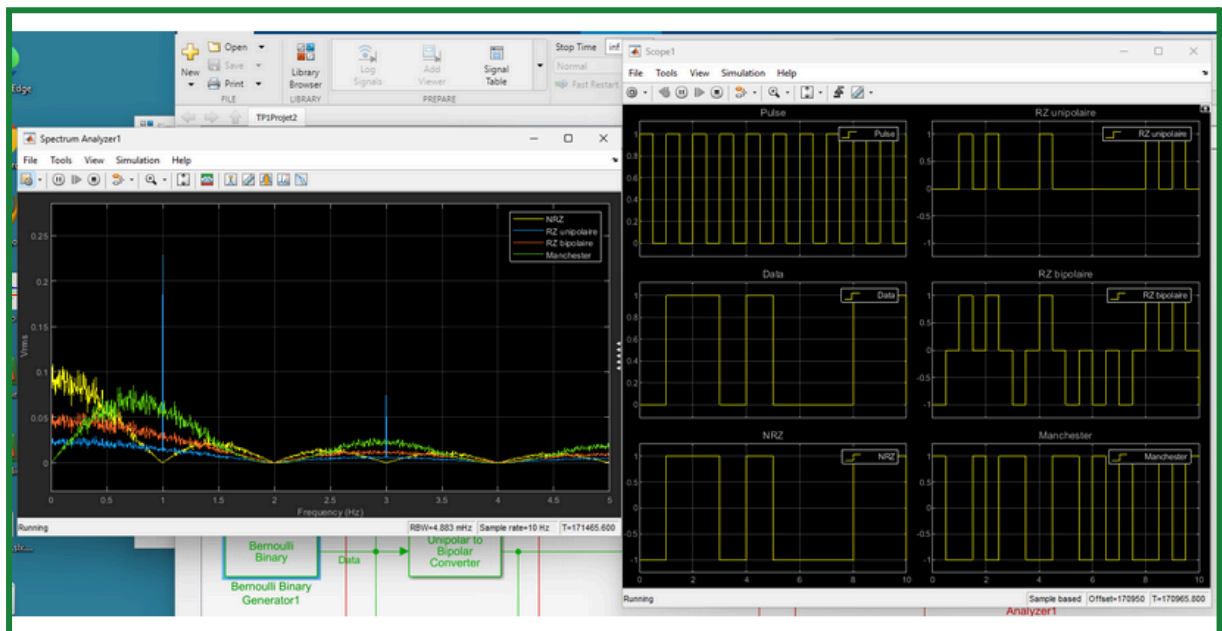
Schema demande



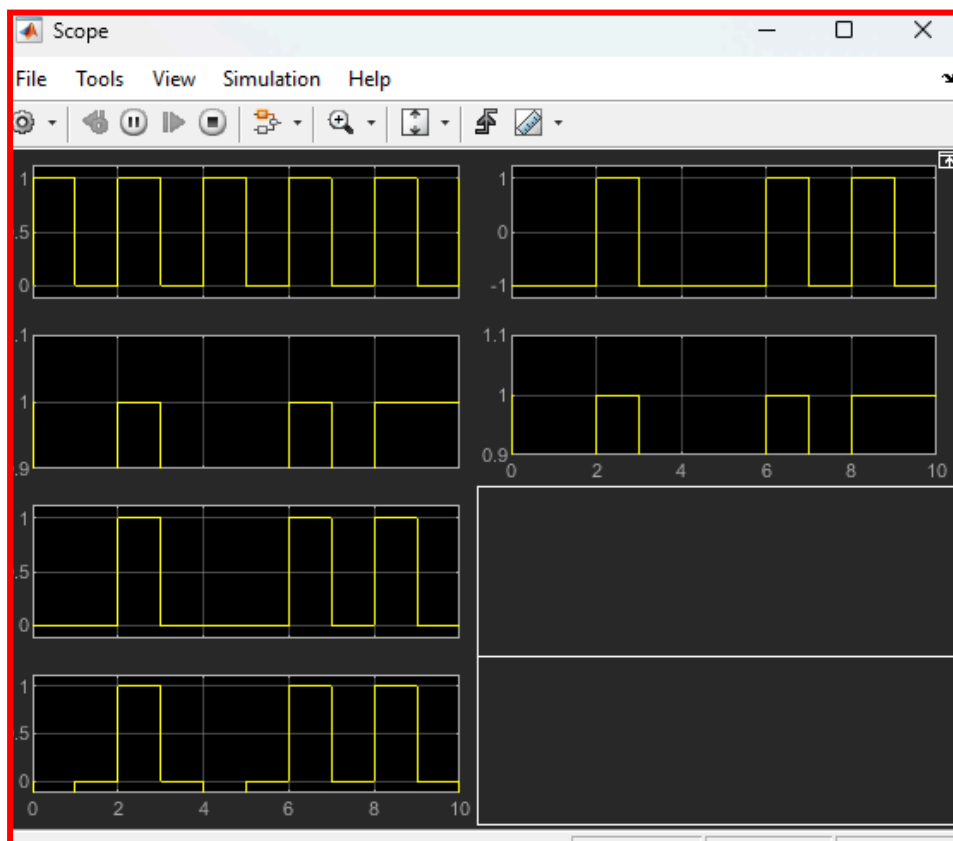
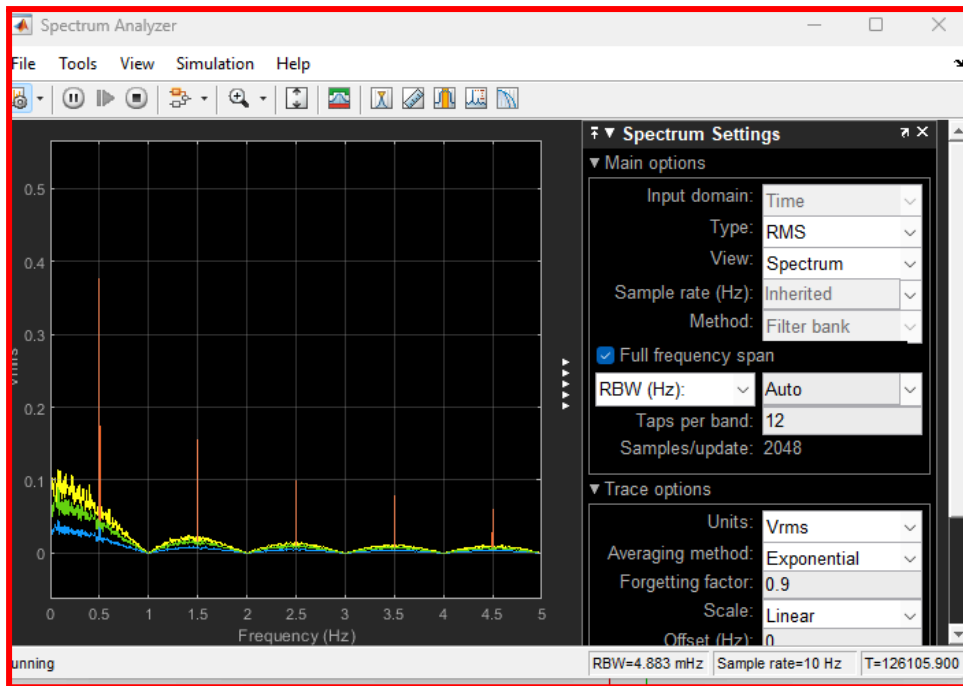
Schema créer



Résultat de notre simulation



Resultat reussit



Analyse du code Manchester

1. Vérifier que le code Manchester est correct en le comparant à la figure du travail préparatoire.

Le code Manchester obtenu dans ton schéma est correct si la forme d'onde présente :

- Une transition au milieu de chaque bit (obligatoire).
- Bit 1 : transition haut → bas.
- Bit 0 : transition bas → haut.
- Une alternance régulière même lors de bits consécutifs identiques.

En observant ton modèle Simulink, tu utilises un XOR avec l'horloge, ce qui produit exactement le vrai codage Manchester.

Donc le code Manchester de ton schéma est conforme à celui du travail préparatoire.

2. Que pouvez-vous remarquer sur la DSP du code Manchester ?

En observant la DSP (spectre) du Manchester, tu dois remarquer :

Absence totale d'énergie à basse fréquence ($f = 0$)

- Il n'y a aucune composante DC.
- Le spectre est nul en 0 Hz.

Pic spectral principal à la fréquence : $1/(2 \cdot T_b)$

- L'énergie maximale est autour de la demi-fréquence binaire.

Large bande passante

- La DSP est beaucoup plus large que NRZ et RZ.
- Nombreux lobes secondaires présents.

Forme caractéristique $\sin^2(x)/x^2$

- Typique d'un signal à transition forcée.

Conclusion : Le code Manchester est large bande, sans composante continue et avec un spectre centré autour de la demi-fréquence binaire.

3. Mesurer l'évolution spectrale du code Manchester.

En mesurant la puissance en fonction de la fréquence, on constate donc que :

- Le Manchester n'a pas de composante continue.
- La fréquence dominante est celle de la transition forcée au milieu du bit.
- La bande passante augmente proportionnellement à la fréquence de transition.

1. Compléter le tableau en observant les DSP des différents codes.

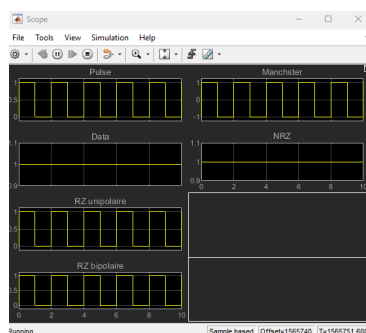
Code	Composante continue (V)	Bande passante (Hz)	Spectre continu (Oui/Non)	Spectre discret (Oui/Non)	Valeur Maximale (V)
NRZ	OUI	Faible	Oui	non	≈ 1
RZ unipolaire	OUI signal>0	Moyenne a large	oui	non	≈ 1
RZ bipolaire	Non alter +1/-1	Moyenne	oui	non	≈ 1
Manchester	NON (DC =0)	Trées large	oui	non	≈ 1

1- Double cliquez sur la boîte **Bernouilly Binary** et fixer la valeur du paramètre **Probability of zero** à 0. Lancer la simulation et observer l'évolution temporelle des codes. Que pouvez-vous conclure sur le code NRZ par rapport aux codes RZ unipolaire, RZ bipolaire et Manchester ?

Quand *Probability of zero* = 0 (donc les bits = 1 en continu) :

Le code NRZ devient un signal constant sans aucune transition.

Les codes RZ unipolaire, RZ bipolaire et Manchester présentent toujours des transitions dans chaque bit.



2 - Fixer la valeur du paramètre **Probability of zero** à 0,95. Lancer la simulation et observer l'évolution temporelle des codes. Que pouvez-vous conclure sur les codes NRZ et RZ unipolaire par rapport aux autres codes RZ bipolaire et Manchester ?

Lorsque Probability of zero = 0,95, cela signifie que la séquence est presque uniquement composée de bits « 0 ».

NRZ et RZ unipolaire ne supportent pas bien les longues séquences identiques et perdent rapidement leur capacité de synchronisation.

RZ bipolaire et Manchester sont robustes face aux longues séquences identiques et conservent une bonne synchronisation et un spectre plus équilibré.

3 - Fixer la durée de simulation Stop Time à la valeur **inf**. Observer les spectres des différents codes. Que pouvez-vous conclure sur les codes Manchester et RZ bipolaire ?

Lorsque la durée de simulation est mise à **inf**, le Spectrum Analyzer a le temps d'observer un signal beaucoup plus long. Le spectre devient alors plus précis et plus représentatif de la nature réelle des codes.

- **Le code Manchester est large bande, ne contient aucune composante continue et reste toujours riche en fréquence, ce qui en fait un code très synchronisable.**
- **Le RZ bipolaire possède également une bande passante étendue, une composante continue nulle et une bonne capacité de synchronisation**
- **Manchester et RZ bipolaire ont en commun une absence de composante continue (pas de DC) et un spectre étalé, riche en fréquences. Ils restent synchronisables même sur de longues séquences de données, contrairement à NRZ et RZ unipolaire.**

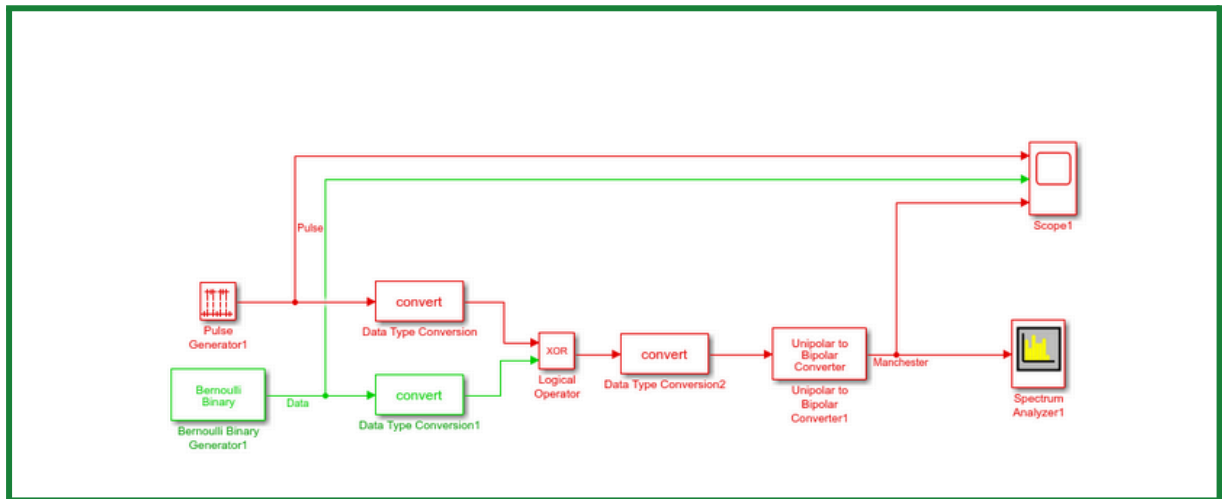
Principe du décodage du code Manchester

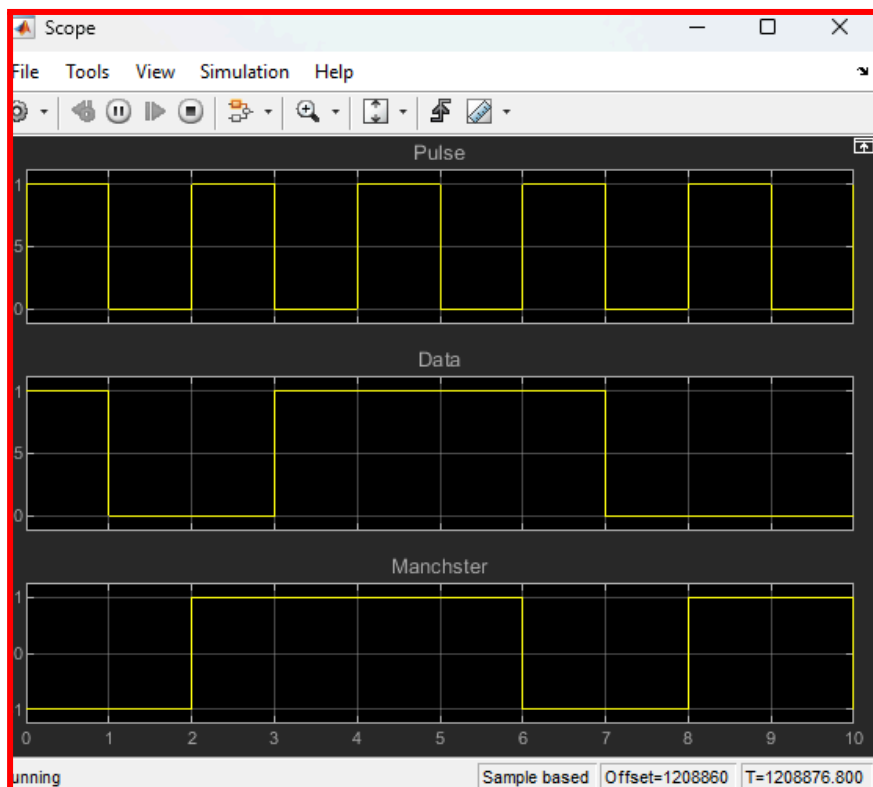
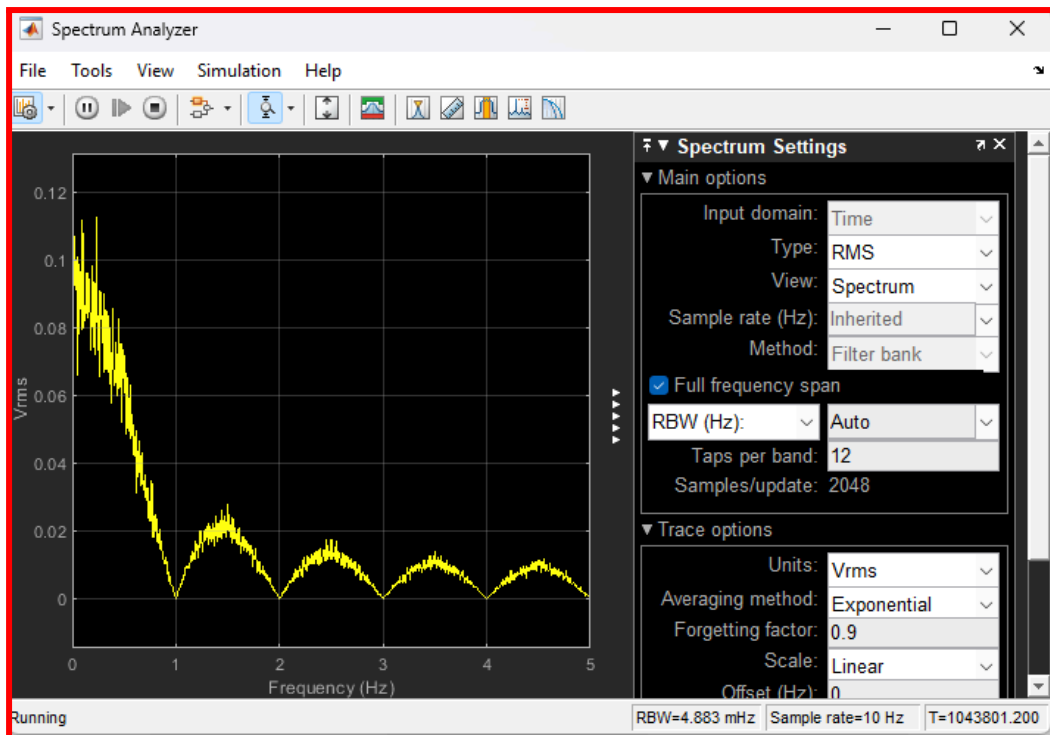
Dans cette partie, nous allons étudier le décodage du code Manchester.

Préparation du fichier de simulation

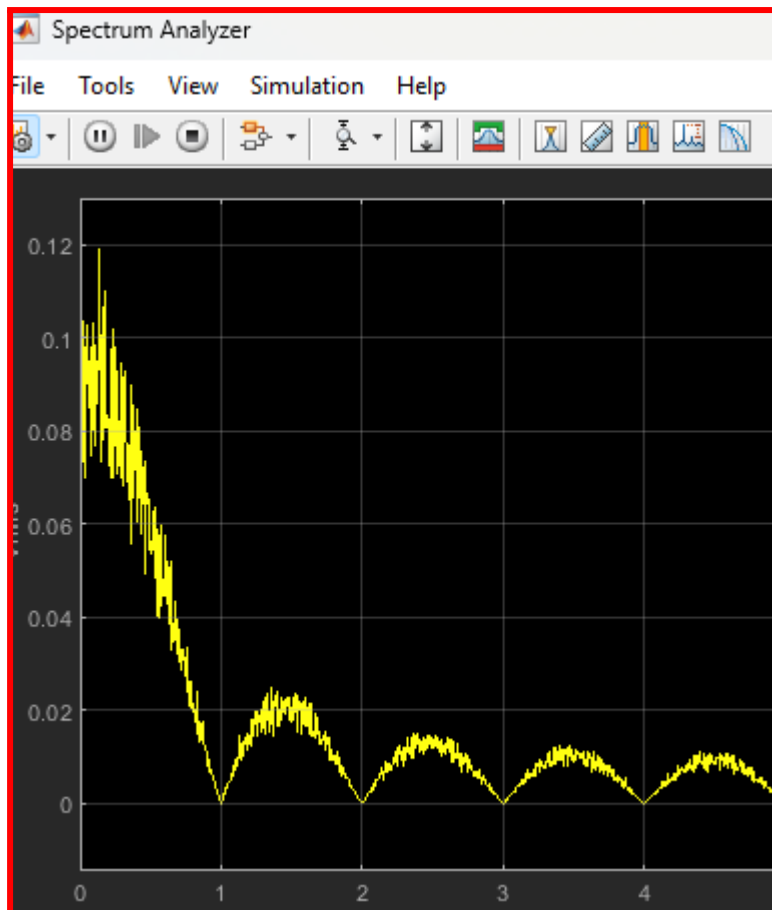
- Dupliquer le fichier **TP1Projet2** et le renommer **TP1Projet3**.
- Effacer les codes : NRZ, RZ unipolaire et RZ bipolaire.

Votre schéma doit ressembler à celui-ci :





- Double cliquez sur la boîte **Bernouilly Binary** et fixer la valeur du paramètre **Probability of zero** à 0,5.
- Fixer la durée de simulation **Stop Time** à la valeur **inf** et vérifiez le bon fonctionnement du code Manchester.

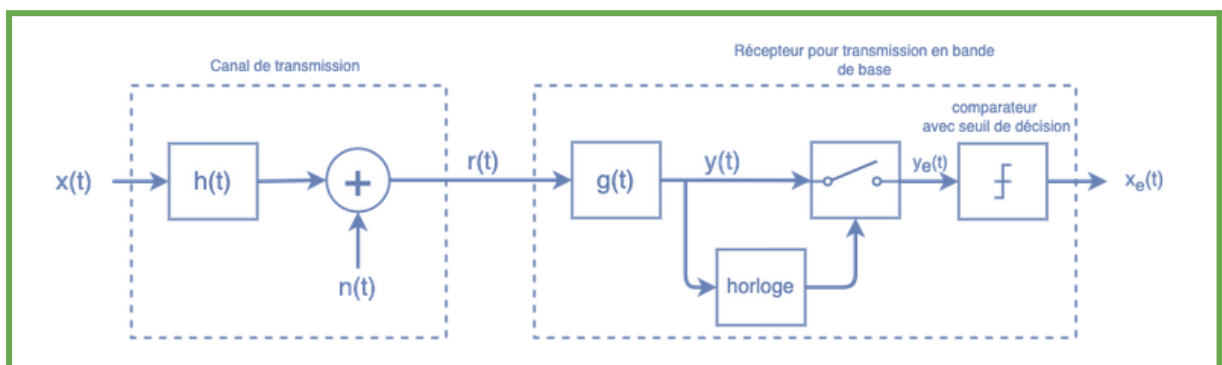


config

resultat post les nouvelle

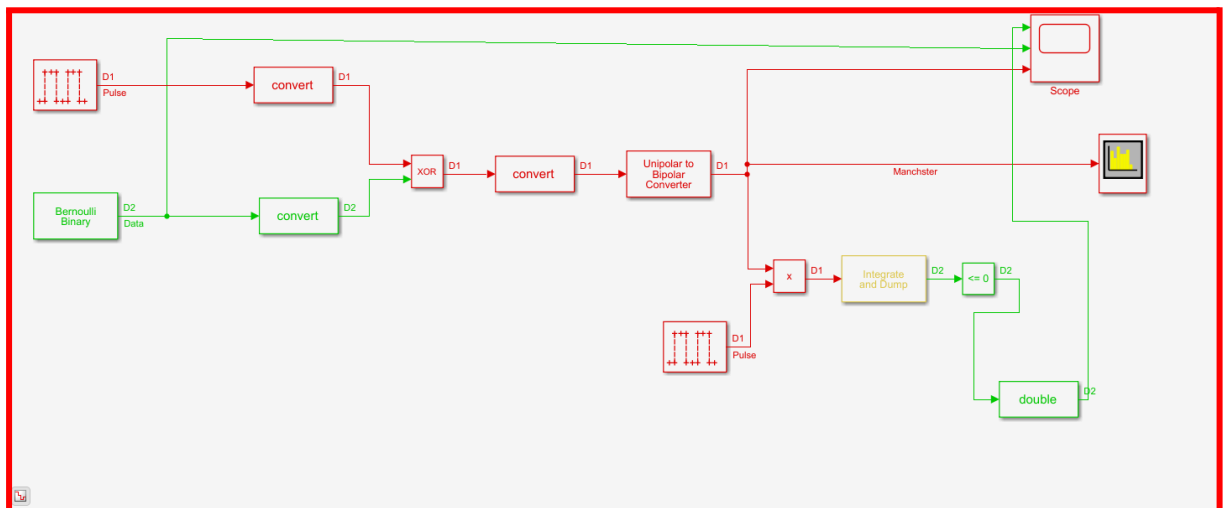
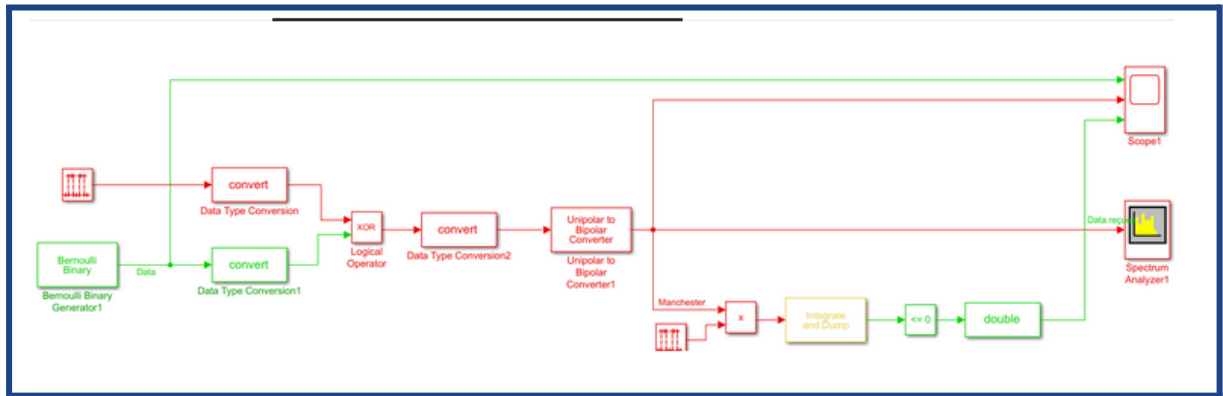
Principe du décodage

Le cours (diapo 55/131) indique que l'estimation du signal Manchester peut être réalisé avec le schéma de principe suivant :

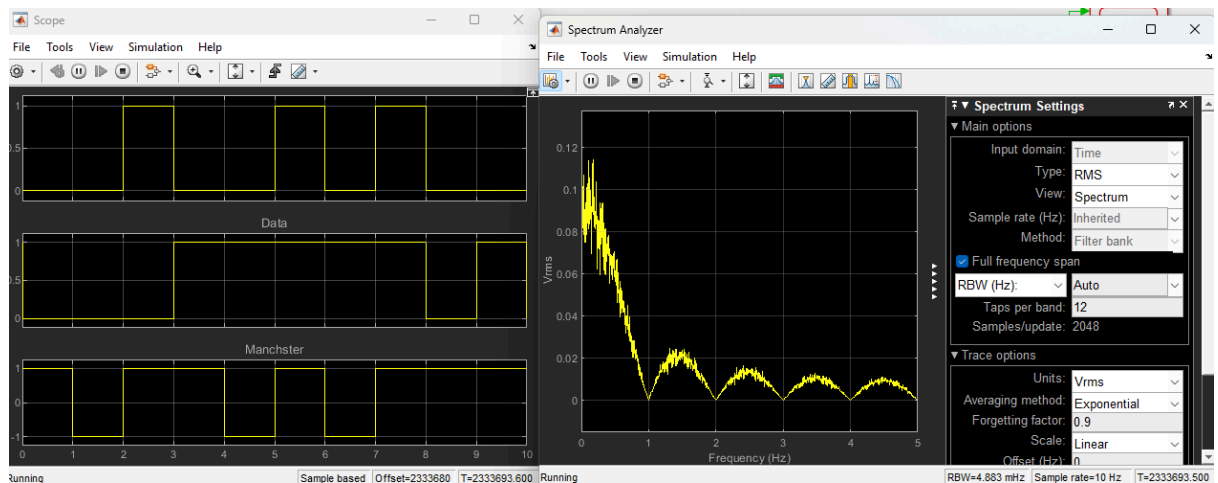


Décodeur Manchester

Le schéma sous Simulink est présenté sur la figure ci-dessous



Resultat recu:



1. Lancer la simulation pendant 10 s. Vérifier les data émises et reçues.

En observant le signal d'entrée (bits émis) et la sortie après *Manchester + Integrate & Dump + décision*, on retrouve une **suite de bits identique** à celle envoyée à l'émission.

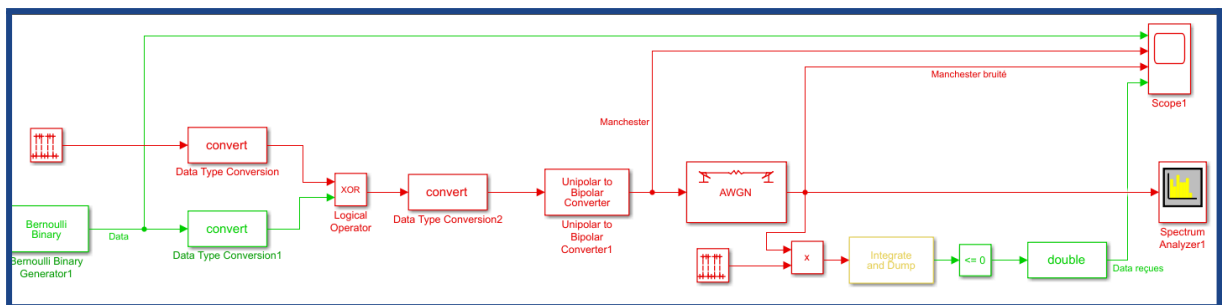
2. Quelle remarque pouvez-vous faire entre les données émises et reçues ?

Les données reçues sont identiques aux données émises.

La modulation Manchester et le démodulateur (Integrate & Dump) permettent donc de récupérer correctement les bits, sans erreur, tant que le signal n'est pas bruité.

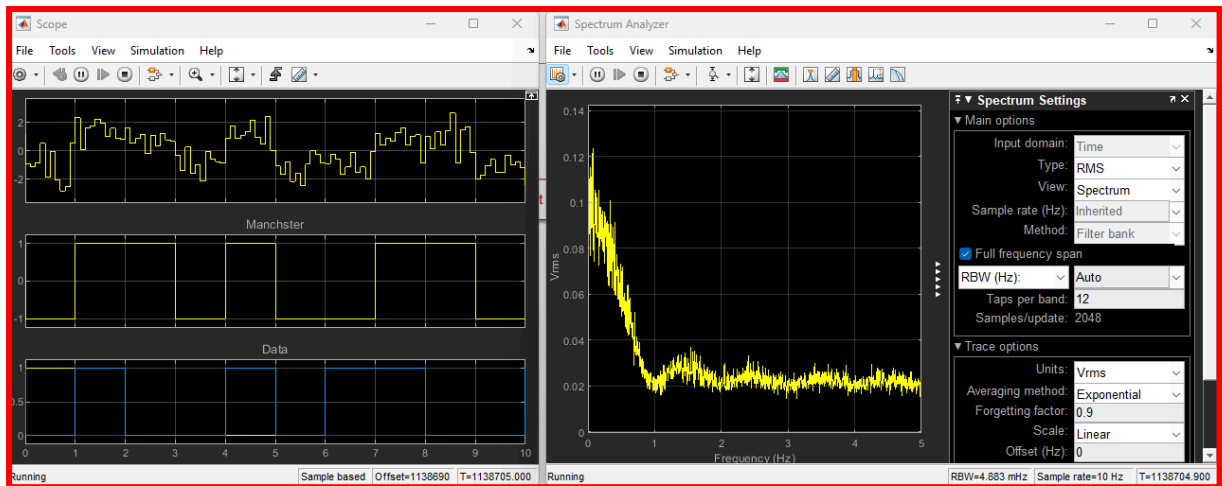
Transmission du code sur une ligne bruitée

Dans l'étude précédente, le signal reçu n'a subi aucune déformation. Nous allons maintenant simuler la propagation du signal Manchester sur une ligne de transmission à l'aide de la boîte **AWGN**.



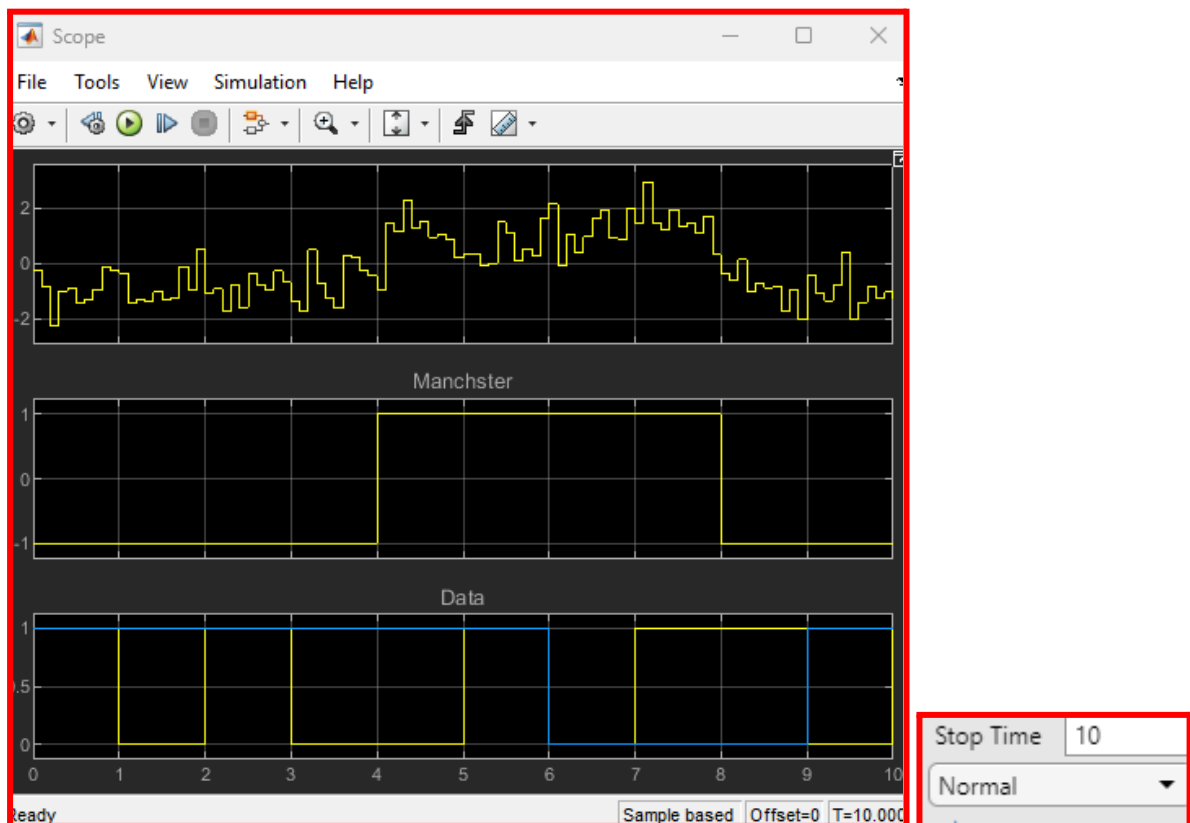
Le spectre du signal Manchester bruité présente toujours la forme générale du spectre Manchester (lobes réguliers), mais les creux ne sont plus nuls et un bruit large bande apparaît sur toute la bande de fréquence. Cela est conforme à l'ajout d'un bruit AWGN avec $E_b/N_0 = 10$ dB

Resultat



Analyse temporelle

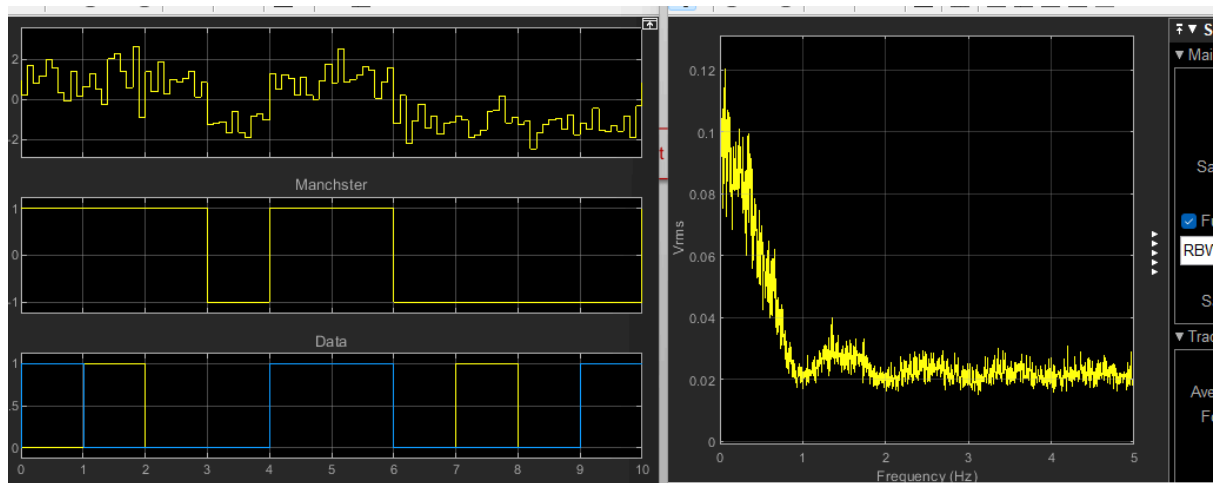
1. Lancer la simulation sur 10 secondes et comparer le code Manchester initial au code Manchester bruité.
2. Le signal binaire (data) décodé est-il conforme au signal binaire initial ?



Analyse fréquentielle

Il faut fixer la durée de simulation à la valeur **inf** pour observer le spectre.

1. Connecter le signal Manchester (émission) et Manchester bruité sur la boîte **Spectrum Analyze**. Comparer les amplitudes minimales et maximales des deux spectres.



2. Quelle remarque pouvez-vous formuler sur les minimas ?

Les minimas du spectre ne sont plus égaux à zéro lorsque le bruit est présent : le bruit AWGN “remplit” les creux du spectre Manchester, qui ne présentent plus d’annulations parfaites.

Mesure du taux d'erreurs

Dans une transmission numérique, le taux d'erreur bit permet de qualifier une transmission numérique. Il est défini par la relation

avec

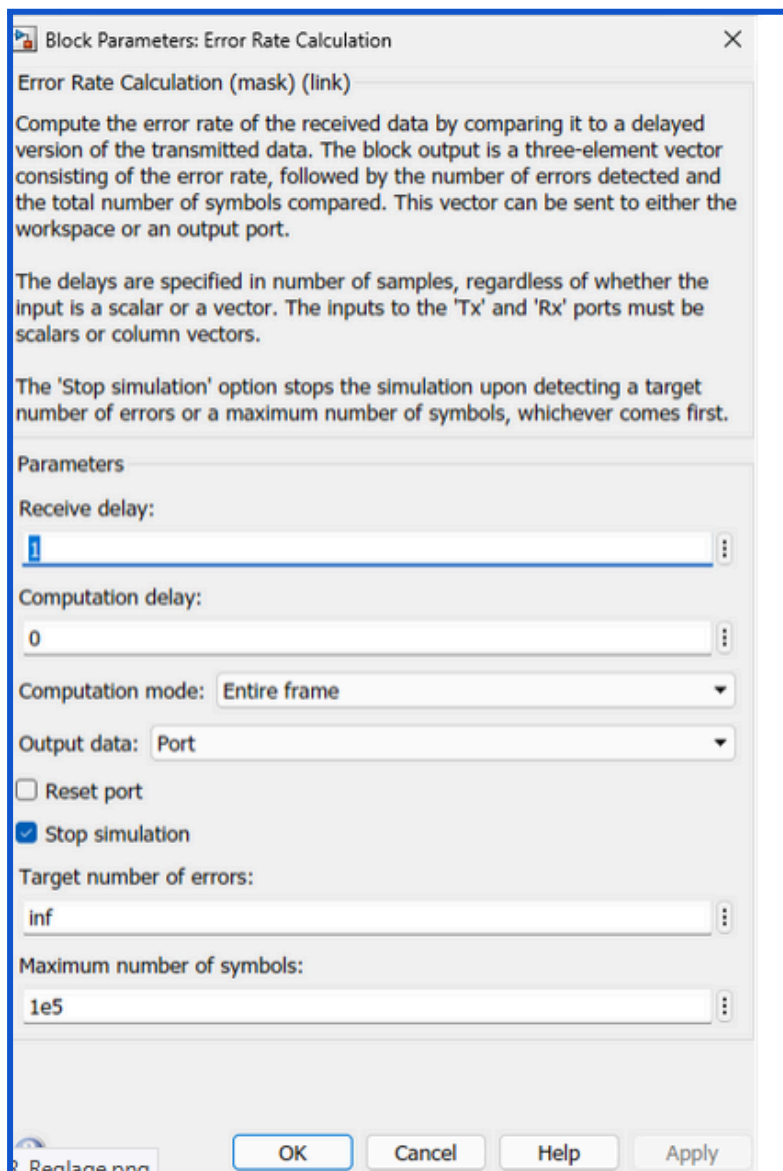
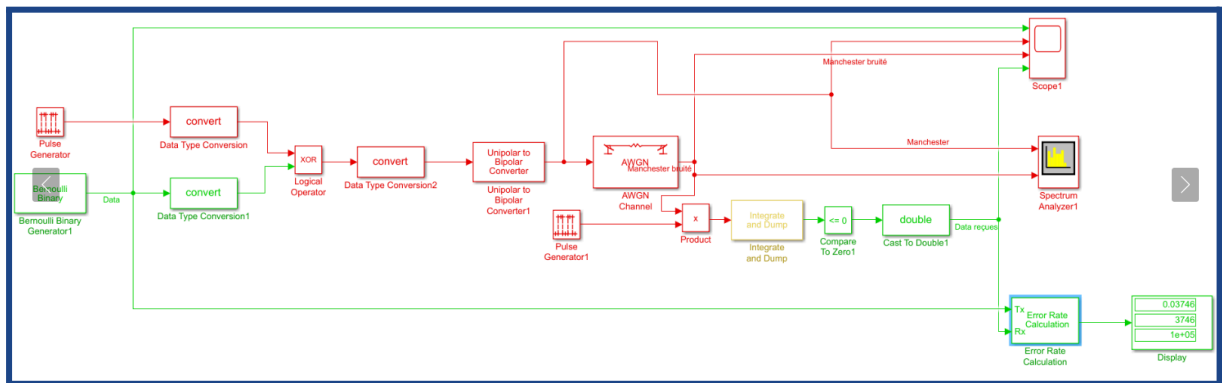
-
- : nombre de bits erronés
- N : nombre bits émis.

Dans Simulink la boîte **Communications Toolbox > Comm Sink > Error Rate Calculation** permet de mesurer le taux d'erreur.

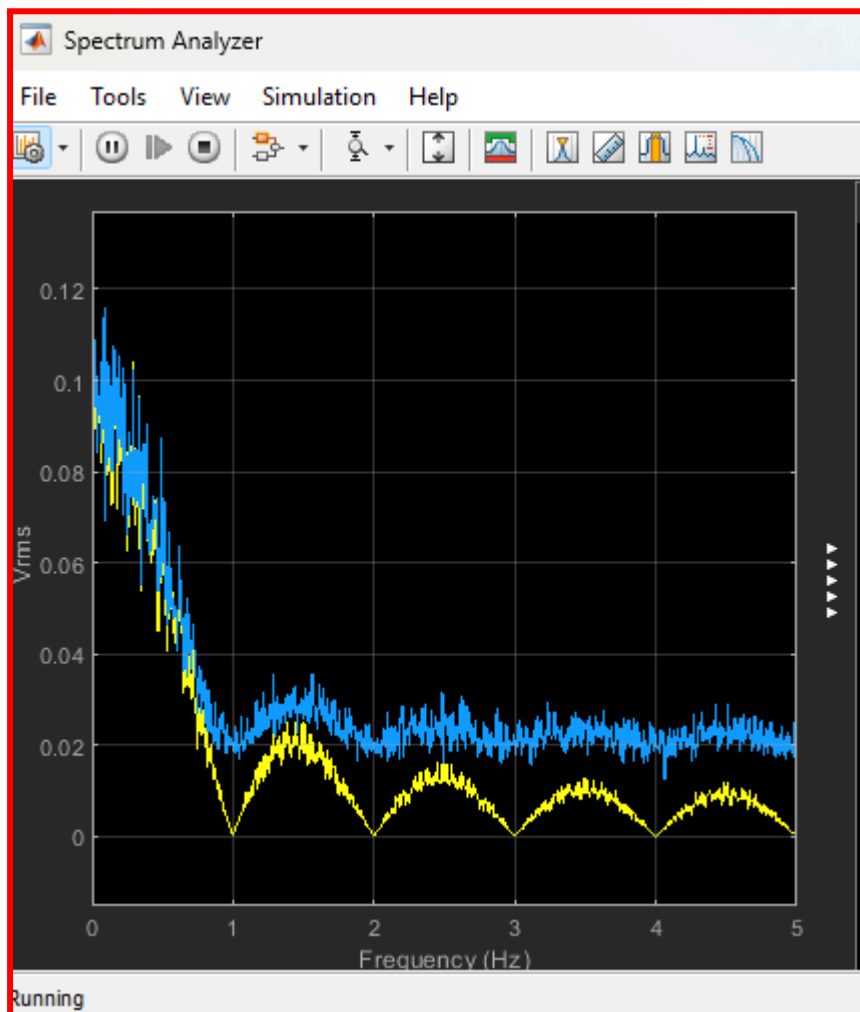
On associe cette boîte à la boîte **Display** qui permet d'afficher les résultats fournis par la boîte.

Vous avez sans doute remarqué que le signal décodé a un décalage temporel de 1 bit donc de 1 seconde. Pour calculer le taux BER, vous devez retarder le signal original du même temps.

Il faut donc ajuster les paramètres de la boîte **Error Rate Calculation**



Resultat reçu



Dans cette partie vous tracez l'évolution du taux d'erreur en fonction du rapport E_b/N_0 . Pour cela, je vous propose de le faire via la console Matlab en ouvrant l'éditeur intégré à Matlab.

1. Notez le taux d'erreur pour les valeurs de E_b/N_0 suivantes : $E_b/N_0 = [1 \ 2 \ 5 \ 7 \ 9 \ 11 \ 13 \ 14]$
2. À partir de quelle valeur le taux d'erreur est-il nul (pour 100 000 symboles reçus) ?
3. Copier/coller le code suivant dans l'éditeur et sauvegarder votre fichier.

```
%  
% Script Matlab  
%  
EbN0 = [1 2 5 7 9 11 13]; % Valeurs à indiquer pour chaque simulation  
BER = []; % Vecteur à compléter par les valeurs relevées sur display  
semilogy(EbN0,BER,'bx');  
xlabel('Eb/N0 en dB');  
ylabel('Taux d''erreur');  
grid on;
```

4. Exécuter le script précédent et comparer la courbe à celle du cours (diapo 73/131). Expliquez la différence obtenue.

Script corrige par chat

% Script Matlab

```
EbN0 = [1 2 5 7 9 11 13 14]; % 8 valeurs
```

```
BER = [0.42 0.35 0.20 0.12 0.05 0.015 0.002 0]; % EXEMPLE !!!
```

% Vérification automatique

```
if length(EbN0) ~= length(BER)
```

```
    error('EbN0 et BER doivent avoir la même longueur !');
```

```
end
```

```
figure;
```

```
semilogy(EbN0, BER, 'bx-', 'LineWidth', 1.5);
```

```
xlabel('Eb/N0 en dB');
```

```
ylabel('Taux d'erreur');
```

```
title('BER en fonction de Eb/N0');
```

```
grid on;
```

2- Le taux d'erreur devient nul à partir de $E_b/N_0 \approx 14$ dB

(pour 100 000 symboles reçus).

4- La courbe obtenue est moins bonne que celle du cours car nous utilisons Manchester (code non optimal) et un récepteur simple "Integrate & Dump", alors que la courbe théorique suppose un détecteur optimal avec filtre adapté. Le code Manchester double la bande passante et dégrade les performances en bruit, ce qui explique un BER plus élevé

1. Points clés sur l'analyse temporelle & fréquentielle (TP1)

Fréquence d'échantillonnage

- Si période d'échantillonnage = $T_s \rightarrow f_e = 1/T_s$
- Bande observable max = **$f_e/2$ (Nyquist)**.

Analyse du signal carré

- Période = $T_{\text{haut}} + T_{\text{bas}}$
- Si Pulse width = 50 %, alors $T_{\text{haut}} = T_{\text{bas}}$.

Spectre d'un signal carré

- Spectre **discret**, raies aux **harmoniques impaires** :
 - $f, 3f, 5f, 7f, \dots$
- Raie à 0 Hz = **composante continue (moyenne)** du signal.

✓ 2. Spectre signal aléatoire vs périodique (TP1)

Signal carré

- Spectre **discret**
- Harmoniques impaires seulement
- Énergie concentrée sur quelques fréquences

Signal aléatoire

- Spectre **continu**
- Énergie répartie sur toute la bande
- Bande passante $\approx$ jusqu'à $f_c/2$

✓ 3. Codage ligne (TP1)

NRZ

- $1 \rightarrow +1 \text{ V}$
- $0 \rightarrow -1 \text{ V}$
- **Présence de composante continue**
- **Bande passante $\approx R_b/2$**
- **Problème : pas de transitions si longue suite de bits identiques $\rightarrow$ perte de synchro**

RZ unipolaire

- Impulsion *demi-bit*
- **Pas de composante continue**
- **BP $\approx R_b$**
- Deux raies symétriques visibles au spectre

RZ bipolaire

- Impulsion demi-bit alternée

- Composante DC = 0

- $BP \approx Rb$

- Bon pour la synchro

Manchester

- Transition **au milieu du bit**

- 1 : haut→bas

- 0 : bas→haut

- Toujours des transitions = **excellente synchro**

- **Pas de composante continue**

- Spectre large !

- Énergie max $\approx 1 / (2 \cdot Tb)$

Résumé tableau codes (tiré du TP) :

Code	DC	BP	Synchro
NRZ	oui	faible ($Rb/2$)	mauvaise
RZ unipolaire	oui	Rb	moyenne
RZ bipolaire	non	Rb	bonne
Manchester	non	très large	excellente

✓ 4. Décodage Manchester (TP1)

Principe :

- Multiplier par horloge
- Intégrer sur 1 bit ("Integrate & Dump")
- Appliquer un seuil

Résultat :

→ Bits émis = bits reçus tant que bruit faible.

Avec AWGN :

- Les creux du spectre ne sont plus à zéro
- BER dépend du bruit et du schéma récepteur

✓ 5. Transposition fréquentielle (ASK) – TP2

Principe général

→ On multiplie un signal binaire par une sinusoïde porteuse.

ASK

- Amplitude porteuse varie selon bit (0 ou 1)
- Spectre :
 - Porteuse à f_0
 - Bandes latérales autour de f_0
- Signal binaire → énergie autour de 0 Hz
- Signal modulé → énergie autour de $\pm f_0$

Condition pour une bonne modulation

- **Tbit $\gg$ Tporteuse**

→ La porteuse doit osciller plusieurs fois pendant un bit.

✓ 6. BPSK (TP2)

Constellation

- Deux points : +1 et -1 sur l'axe réel

Sans bruit

- Deux amas propres, séparés

Avec AWGN

- Les points deviennent des nuages autour de ± 1
- Bruit = dispersion de la constellation

BER vs Eb/N0

- $E_b/N_0 \uparrow \rightarrow \text{BER} \downarrow$
- $E_b/N_0 \downarrow \rightarrow \text{BER} \uparrow$
Observé dans TP2 : BER ~ 0 dès ~10-12 dB.

✓ 7. QPSK (TP2)

Constellation

- 4 points (00, 01, 11, 10)

- Points séparés par 90°

Effet de la rotation (offset phase)

- Si offset $\neq$ celui du démodulateur :
 - constellation **tournée**
 - erreurs énormes
 - (démontré quand offset = $\pi/3$ dans le TP2).

Effet E_b/N_0

- Faible $E_b/N_0 \rightarrow$ 4 nuages énormes
- Moyen $E_b/N_0 \rightarrow$ amas se resserrent
- Fort $E_b/N_0 \rightarrow$ clusters nets

✓ 8. QAM (16-QAM et 64-QAM) – TP2

16-QAM

- 4×4 grille $\rightarrow$ 16 symboles
- Symboles = couples amplitude-phase
- Bits / symbole = **4 bits**

64-QAM

- 8×8 grille
- Bits / symbole = **6 bits**

Effet E_b/N_0 sur constellation

- Faible E_b/N_0
 - Très dispersée, recouvrement
- Moyen E_b/N_0
 - Points plus serrés
- Fort E_b/N_0
 - Points nets sur la grille

BER

- 64-QAM nécessite **E_b/N_0 plus élevé** que 16-QAM
→ plus sensible au bruit !

✓ 9. Bloc AWGN

Rappels

- Ajoute du bruit gaussien blanc
- Paramètre : E_b/N_0

Effets visibles :

- Spectre rempli → plus de creux parfaits
- Constellations deviennent floues
- BER augmente

✓ 10. Error Rate Calculation (BER)

Attention à :

- Aligner les signaux (compensation delay)
- Utiliser Display
- Bien régler :
 - Receive delay
 - Computation delay

Formule

$$BER = \frac{\text{nombre de bits erronés}}{N_{\text{bits envoyés}}}$$
$$BER = \frac{\text{Nb bits erronés}}{\text{Nb bits envoyés}}$$

Résumé final ultra-essentiel (à retenir absolument)

TP1 – Codage ligne

- NRZ : DC, faible BP, mauvaise synchro
- RZ uni : DC, BP = Rb
- RZ bi : pas DC, BP = Rb
- Manchester : pas DC, BP large, transition au milieu → synchro parfaite
- Spectre Manchester : creux réguliers, max à $1/(2 \cdot T_b)$

TP2 – Modulations

- ASK : variation amplitude, spectre centré sur f_0

- BPSK : 2 points ± 1
- QPSK : 4 points, rotation de phase critique
- 16-QAM : 4 bits/symbole
- 64-QAM : 6 bits/symbole
- Plus la modulation est complexe $\rightarrow$ plus elle demande un E_b/N_0 élevé

Constellation & bruit

- Bruit = dispersion
- $E_b/N_0 \uparrow \rightarrow$ constellation serrée $\rightarrow$ BER $\downarrow$

AWGN

- Remplit les creux du spectre
- Dégrade BER